

Open AI Cubes

Student:

Santa Cruz Basilio, Jesus Diego
Serrano Arostegui, Max Jairo
Pumayauli Evangelista, Bryan José
López Arteaga, César Omar

Universidad Nacional de Ingeniería, Facultad de Ciencias
e-mail: jesus.santacruz.b@uni.pe, max.serrano.a@uni.pe
bryan.pumayauli.e@uni.pe, clopeza@uni.pe

Curso:

Programación Concurrente y Distribuida - CC4P1A

Proyecto Parcial

Abstract

Open AI Cubes es un concepto arquitectónico para construir sistemas de Inteligencia Artificial modulares. Cada cubo representa una capacidad especializada de IA en una dimensión determinada, por ejemplo texto, imagen, audio o señales. El proyecto implementado materializa este concepto mediante una versión funcional con dos dimensiones principales: procesamiento de texto y procesamiento de imágenes.

La solución se implementó usando exclusivamente el SDK estándar de Java, sin librerías externas. La arquitectura adopta un esquema cliente-servidor TCP multiciente. El servidor centraliza los modelos de IA, carga los archivos binarios entrenados y atiende múltiples clientes mediante hilos independientes. El cliente no carga modelos; únicamente envía solicitudes y muestra las respuestas recibidas.

Keywords: Dijkstra, backpropagation, paralelización, concurrencia, rendimiento.

Contents

1	Introducción	3
2	Marco conceptual: Open AI Cubes y Micro Chunk	3
2.1	Open AI Cubes	3
2.2	Micro Chunk	3
2.3	Arquitectura general del sistema	3
2.4	Módulo de procesamiento de imágenes	4
2.5	Módulo de procesamiento de texto	5
3	Protocolo de comunicación	6
4	Implementación de los Clientes	8
4.1	Implementación de Cliente - Java	8
4.1.1	Arquitectura de Red (TCPClient50)	8

4.1.2	Interfaz de Usuario Modular (MainMenuCliente)	8
4.1.3	Cubo de Procesamiento de Imagen (MainFormularioMNISTCliente)	9
4.1.4	Cubo de Procesamiento de Texto(MainFormularioWord2VecCliente)	9
4.1.5	Diagrama de Clases: Arquitectura del Cliente	10
4.2	Implementación de Cliente - Python	11
4.2.1	Capa de Transporte: <code>tcp_client_50.py</code>	11
4.2.2	Procesamiento de Imágenes: <code>form_mnist.py</code>	13
4.2.3	Procesamiento de Lenguaje Natural (<code>form_w2v.py</code>)	14
4.2.4	Orquestación y Lanzamiento (<code>cliente_main.py</code> y <code>__main__.py</code>)	15
4.2.4.1	Función: <code>iniciar()</code>	15
4.2.5	Navegación Central (<code>main_menu_cliente.py</code>)	16
4.3	Implementación de Cliente - C++	16
5	Pruebas y Resultados	20
5.1	Resultados de pruebas en maquina en Paralelo	20
5.2	Resultados de prueba en maquinas Distribuidos	23
6	Conclusiones	26

1 Introducción

Los sistemas actuales de IA tienden a integrar varias capacidades: comprensión de texto, clasificación de imágenes, procesamiento de audio, búsqueda semántica y toma de decisiones. Para evitar arquitecturas monolíticas difíciles de mantener, se propone organizar el sistema en cubos funcionales. Cada cubo encapsula una capacidad específica y expone una interfaz de consumo que puede ser invocada por otros componentes.

En este proyecto describe el diseño y la implementación de Open AI Cubes. La solución conserva una estructura modular: una etapa de entrenamiento, un servidor Micro Chunk y un cliente Micro Chunk. El servidor actúa como punto de orquestación de los cubos de IA y permite que varios clientes consuman las capacidades de texto e imagen de forma concurrente.

2 Marco conceptual: Open AI Cubes y Micro Chunk

2.1 Open AI Cubes

Open AI Cubes se entiende como una arquitectura donde cada cubo encapsula una dimensión de inteligencia artificial. Un cubo no es solamente una clase, sino una unidad funcional que puede incluir preprocesamiento, modelo, inferencia, serialización de resultados y comunicación con otros módulos.

Table 1: Cubos considerados en la arquitectura

Cubo	Dimensión	Capacidad
Cubo de texto	Texto	Embeddings, similitud semántica y analogías vectoriales.
Cubo de imagen	Imagen	Normalización de imagen, clasificación de dígitos y probabilidades por clase.
Cubo futuro	Audio, sensores, video u otro	Extensión del protocolo del servidor para incorporar nuevas capacidades.

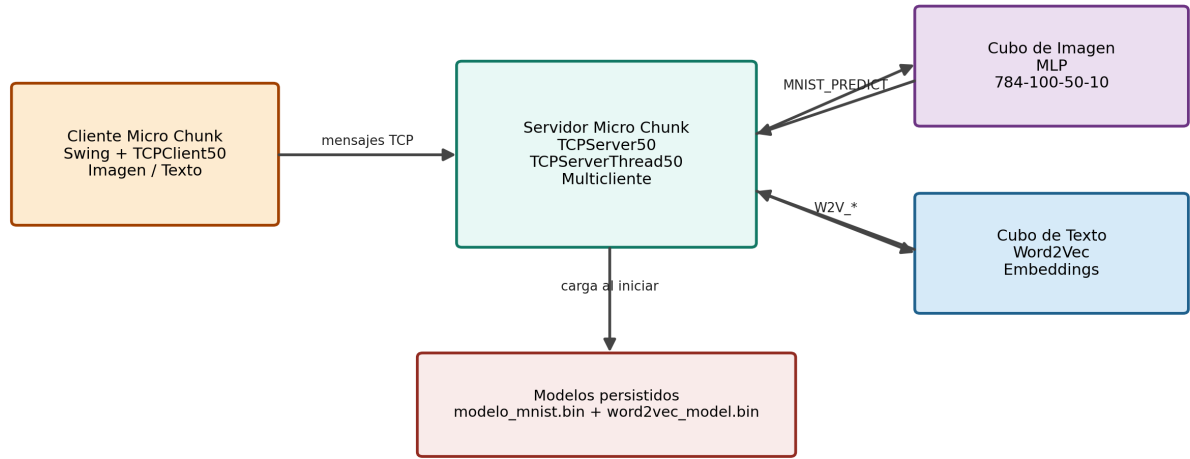
2.2 Micro Chunk

Un Micro Chunk es una unidad pequeña de cómputo que puede ejecutarse de forma concurrente, paralela o distribuida. En el proyecto, los micro chunks principales son los modelos y los trabajadores de red: MLP, Word2VecModel, TCPServer50 y TCPServerThread50. Estos componentes permiten procesar solicitudes de forma separada sin mezclar la lógica del cliente con la lógica de inferencia.

2.3 Arquitectura general del sistema

La arquitectura implementada separa tres zonas: entrenamiento, servidor de inferencia y cliente. La etapa de entrenamiento genera los archivos binarios de los modelos. El servidor los carga y expone operaciones por TCP. El cliente ofrece formularios gráficos y envía mensajes al servidor.

Arquitectura general de Open AI Cubes



El cliente es ligero; el servidor centraliza la inferencia y atiende múltiples clientes en paralelo.

Figure 1: Arquitectura general del sistema Open AI Cubes.

Table 2: Capas principales de la solución

Capa	Responsabilidad	Clases o archivos principales
Cliente	Captura la entrada, convierte la imagen, envía solicitudes al servidor y muestra la respuesta.	<code>Cliente50</code> , <code>TCPClient50</code> , <code>MainMenuCliente</code> , <code>MainFormularioMNIST</code> .
Servidor	Carga los modelos, administra múltiples clientes y ejecuta la inferencia solicitada.	<code>Servidor50</code> , <code>TCPServer50</code> , <code>TCPServerThread50</code> .
Entrenamiento	Genera los modelos binarios usados posteriormente por el servidor.	<code>MainEntrenarGuardar</code> , <code>MNISTLoader</code> , <code>MainEntrenarWord2Vec</code> .

2.4 Módulo de procesamiento de imágenes

El módulo de imagen implementa una red neuronal multicapa o MLP. El cliente trabaja con imágenes a escala de grises, de tamaño 28×28 píxeles y genera un vector de 784 valores normalizados en el intervalo $[0, 1]$. Ese vector se envía al servidor usando el comando `MNIST_PREDICT`.

Table 3: Diseño del módulo de imagen

Elemento	Detalle
Entrada	Vector real de 784 componentes, correspondiente a una imagen de 28×28 píxeles.
Arquitectura	Red neuronal multicapa con capas ocultas de 100 y 50 neuronas, y capa de salida de 10 clases.
Activación	Función ReLU en las capas ocultas y función softmax en la capa de salida.
Entrenamiento	5 épocas, tasa de aprendizaje 0.01, conjunto MNIST de 60 000 muestras de entrenamiento y 10 000 muestras de prueba.
Persistencia	Formato binario con cabecera <code>MNIST_MLP_V1</code> y matrices de pesos W_1 , W_2 , W_3 .

2.5 Módulo de procesamiento de texto

El módulo de texto implementa un modelo Word2Vec de tipo Skip-gram. Su objetivo es aprender representaciones vectoriales de palabras, de modo que palabras semánticamente relacionadas queden cercanas en el espacio vectorial. El modelo soporta dos operaciones principales: búsqueda de palabras cercanas y analogía vectorial.

Table 4: Diseño del módulo de texto

Elemento	Detalle
Modelo	Word2Vec Skip-gram con negative sampling.
Dimensión	100 componentes por vector de palabra.
Ventana	Ventana dinámica entre 1 y <code>window</code> .
Épocas	80 épocas en la configuración del entrenamiento.
Operaciones	<code>nearestWords(palabra, topK)</code> y <code>analogy(A, B, C, topK)</code> .
Persistencia	Formato binario con cabecera <code>WORD2VEC_SKIPGRAM_V1</code> , vocabulario y matrices de embeddings.

3 Protocolo de comunicación

El protocolo de comunicación constituye el mecanismo mediante el cual los clientes Micro Chunk interactúan con el servidor central del sistema Open AI Cubes. Su función principal es permitir que diferentes módulos clientes envíen solicitudes de inferencia hacia el servidor, indicando qué dimensión de inteligencia artificial desean utilizar, por ejemplo texto o imagen, y recibiendo como respuesta el resultado producido por el cubo correspondiente.

En esta implementación, la comunicación se realiza mediante una arquitectura cliente-servidor basada en sockets TCP. El uso de TCP permite establecer una conexión confiable entre el cliente y el servidor, garantizando que los mensajes enviados lleguen en orden y sin pérdida de datos durante la transmisión. Esto es importante porque las solicitudes pueden contener información sensible para el procesamiento, como vectores numéricos, rutas de imágenes, texto de entrada o comandos asociados a operaciones de inferencia.

El servidor actúa como un nodo central encargado de escuchar conexiones entrantes. Cuando un cliente se conecta, el servidor crea un hilo de atención independiente para procesar sus solicitudes. De esta manera, el sistema puede atender múltiples clientes de forma concurrente, evitando que una petición bloquee el procesamiento de las demás. Esta estrategia permite representar el concepto de Micro Chunk distribuido, donde varios clientes pueden consumir capacidades de inteligencia artificial alojadas en un servidor común.

El flujo general del protocolo es el siguiente:

1. El cliente inicia una conexión TCP hacia el servidor indicando la dirección IP y el puerto de comunicación.
2. El servidor acepta la conexión y asigna un hilo de atención para ese cliente.
3. El cliente envía una solicitud estructurada indicando el tipo de operación que desea realizar.
4. El servidor interpreta la solicitud y selecciona el cubo de IA correspondiente.
5. El servidor ejecuta el procesamiento usando el modelo previamente cargado en memoria.
6. El servidor envía una respuesta al cliente con el resultado obtenido.
7. El cliente recibe la respuesta y la muestra en la interfaz o en la consola.

Así cada mensaje del protocolo contiene tres partes fundamentales: el identificador del módulo, la operación solicitada y la carga útil. El identificador del módulo permite diferenciar si la solicitud pertenece al cubo de texto, al cubo de imagen u otro cubo futuro. La operación indica la acción específica que se desea ejecutar, como clasificación de imagen, búsqueda de palabras cercanas o analogía vectorial. Finalmente, la carga útil contiene los datos necesarios para procesar la solicitud.

Table 5: Estructura lógica de una solicitud del protocolo

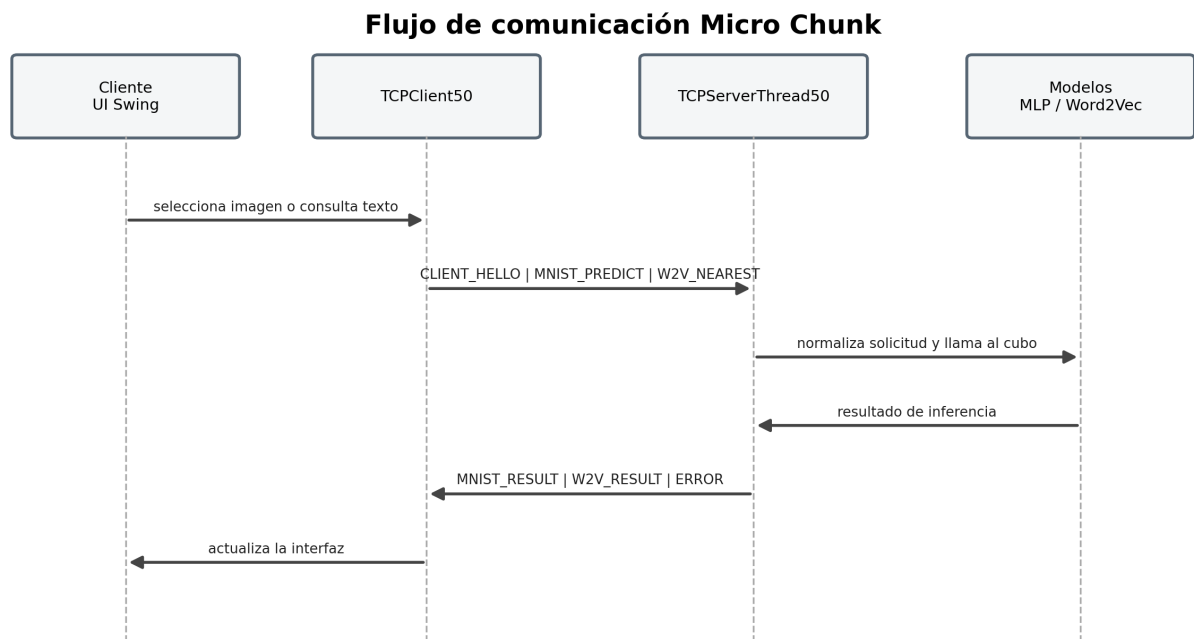
Campo	Descripción
Módulo	Indica el cubo de inteligencia artificial que debe procesar la solicitud. Por ejemplo: texto o imagen.
Operación	Define la acción que se desea ejecutar dentro del módulo seleccionado. Por ejemplo: clasificar una imagen, buscar palabras similares o resolver una analogía.
Carga útil	Contiene los datos enviados por el cliente. Puede ser un vector de imagen normalizado, una palabra, tres palabras para una analogía o cualquier entrada requerida por el modelo.
Respuesta	Resultado devuelto por el servidor después de ejecutar la inferencia. Puede contener una clase predicha, probabilidades, palabras cercanas o mensajes de error.

Para el módulo de imagen, el protocolo permite enviar una imagen previamente convertida a un vector numérico de 784 componentes, correspondiente a una imagen de 28×28 píxeles. El servidor recibe dicho vector, lo entrega al modelo MLP entrenado y obtiene como resultado la clase más probable entre los diez dígitos posibles. La respuesta incluye la predicción generada por el modelo, permitiendo que el cliente muestre el dígito reconocido.

Para el módulo de texto, el protocolo permite enviar consultas asociadas al modelo Word2Vec. Una solicitud puede pedir palabras semánticamente cercanas a una palabra dada o resolver una analogía del tipo $A : B :: C : ?$. En este caso, el servidor consulta el modelo de embeddings cargado en memoria y devuelve una lista de palabras candidatas ordenadas según su similitud vectorial.

El protocolo también contempla el manejo de errores. Si el cliente envía una operación no reconocida, un formato inválido o una entrada incompatible con el modelo solicitado, el servidor debe responder con un mensaje de error en lugar de cerrar abruptamente la conexión.

Así el protocolo de comunicación permite integrar los diferentes componentes de Open AI Cubes en una arquitectura distribuida. El cliente se encarga de capturar y enviar la entrada, mientras que el servidor centraliza la carga de modelos y la ejecución de inferencias. La combinación de sockets TCP, atención concurrente por hilos y mensajes estructurados permite que el sistema sea modular, extensible y capaz de atender múltiples solicitudes en paralelo.



Cada cliente conectado posee su propio hilo en el servidor; los modelos son recursos compartidos protegidos durante la inferencia.

Figure 2: Flujo de comunicación entre cliente, capa TCP y modelos.

4 Implementación de los Clientes

4.1 Implementación de Cliente - Java

La implementación del cliente se ha desarrollado íntegramente en Java, utilizando el entorno de interfaces gráficas Swing y la API de red estándar java.net. El diseño sigue el principio de Micro Chunk, donde el cliente actúa como un consumidor ligero que delega el procesamiento pesado a un servidor centralizado.

4.1.1 Arquitectura de Red (TCPClient50)

El núcleo del cliente es la clase TCPClient50, la cual gestiona una conexión TCP persistente. A diferencia de clientes tradicionales, este componente mantiene el socket abierto para permitir múltiples consultas sin la latencia de reconexión.

Concurrencia y Multihilo: La escucha de mensajes del servidor se ejecuta en un hilo independiente llamado Cliente-TCP-MicroChunk, garantizando que la interfaz de usuario no se bloquee durante la comunicación.

Manejo de Eventos (Callbacks): Se implementó la interfaz OnMessageReceived. Esto permite que cualquier formulario (MNIST o Word2Vec) se suscriba a la red y reciba datos de forma asíncrona.

Seguridad en Hilos: Se utiliza CopyOnWriteArrayList para almacenar los listeners, lo que permite añadir o quitar ventanas de IA en tiempo de ejecución sin generar excepciones de concurrencia.

4.1.2 Interfaz de Usuario Modular (MainMenuCliente)

El acceso a las capacidades de IA se organiza a través de un menú principal que permite seleccionar el "Cubo" o dimensión de procesamiento requerida.

Estado de Conexión: La interfaz muestra en tiempo real la IP y el puerto del servidor al que está vinculado.

Navegación: Utiliza un sistema de visibilidad controlada (setVisible) para alternar entre el menú y los módulos específicos, manteniendo siempre una única instancia del cliente de red activa.

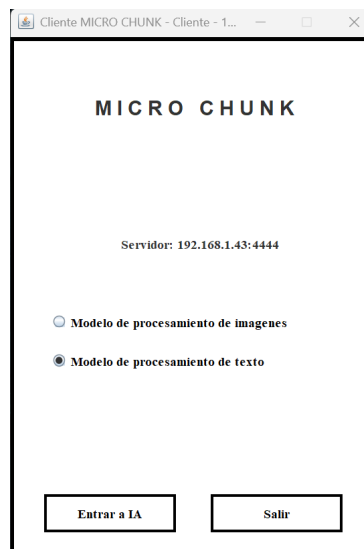


Figure 3: Menú principal del sistema Micro Chunk para la selección de modelos de IA

4.1.3 Cubo de Procesamiento de Imagen (MainFormularioMNISTCliente)

Este módulo permite la clasificación de dígitos manuscritos mediante una red neuronal MLP alojada en el servidor. **Preprocesamiento Local:** El cliente es responsable de normalizar la entrada. La imagen cargada se redimensiona a 28×28 píxeles y se convierte a escala de grises. **Vectorización:** Se genera un vector de 784 valores normalizados en el rango $[0, 1]$. Este vector se serializa en una cadena CSV antes de ser transmitido. **Visualización de Resultados:** Al recibir la etiqueta MNISTRESULT, la UI se actualiza mediante `SwingUtilities.invokeLater()` para mostrar la predicción y el gráfico de probabilidades.



Figure 4: Clasificador MNIST

4.1.4 Cubo de Procesamiento de Texto(MainFormularioWord2VecCliente)

Implementa la interfaz para el modelo semántico Word2Vec, permitiendo realizar operaciones de álgebra vectorial sobre palabras. **Resolución de Analogías:** Permite enviar tres palabras (A, B, C) para que el servidor resuelva la relación $A - B + C \approx ?$. El comando enviado es `W2VANALOGY`. **Búsqueda de Similitud:** El usuario puede consultar las K palabras más cercanas a un término específico mediante el comando `W2VNEAREST`. **Sanitización de Datos:** Se implementó el método `safe()` para limpiar caracteres especiales que pudieran interferir con el protocolo de comunicación basado en tuberías.

Modelo Word2Vec - Cliente

Analogía: A - B + C

A: brasil B: brasilia C: españa

Ejemplo: rey - hombre + mujer = reina

Palabras mas cercanas

Palabra:

brasil - brasilia + españa = ?

1. alemania	similitud = 0.4511393280
2. uruguay	similitud = 0.4197529827
3. mexico	similitud = 0.4146136487
4. producto	similitud = 0.3588318305
5. castilla	similitud = 0.3577412834
6. chile	similitud = 0.3474419648
7. iberia	similitud = 0.3394917893
8. mapa	similitud = 0.3279556329
9. italia	similitud = 0.3261812884
10. muro	similitud = 0.3055854670

Estado: respuesta recibida del servidor

Figure 5: Modulo de texto ejecutando consultas de similitud semantica y analogias

4.1.5 Diagrama de Clases: Arquitectura del Cliente

El diagrama ilustra una arquitectura modular y desacoplada basada en el patrón de diseño Observer, diseñada para gestionar la comunicación asíncrona con el servidor de IA.

Núcleo de Comunicación (TCPClient50): Es el componente central encargado de la persistencia de la red. Gestiona el socket TCP en un hilo independiente y mantiene una lista concurrentes de suscriptores (listeners) para distribuir las respuestas del servidor sin bloquear la interfaz de usuario.

Contrato de Eventos (OnMessageReceived): Esta interfaz define el protocolo de comunicación interna. Permite que cualquier módulo del cliente reaccione a eventos de conexión, desconexión o recepción de mensajes de forma estandarizada.

Especialización de Cubos (Formularios): Las clases MainFormularioMNISTCliente y MainFormularioWord2VecCliente actúan como consumidores especializados. Mediante clases anónimas, implementan la lógica específica para procesar sus respectivas tramas de datos (etiquetas MNISTRESULT o W2VRESULT).

Jerarquía de Interfaz: Se observa una estructura de herencia desde MainMenuCliente (basada en JFrame), lo que permite una gestión centralizada de la navegación y asegura que todos los módulos compartan la misma instancia activa del cliente de red (mTcpClient).

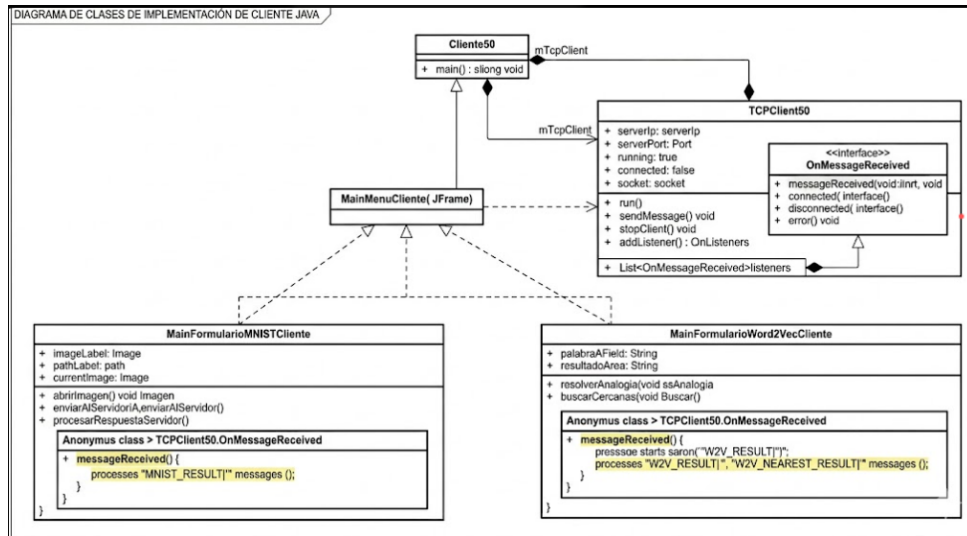


Figure 6: Este diagrama muestra la relacion y las interfaces de los Cubes

4.2 Implementación de Cliente - Python

Para la implementación del cliente en Python, se han utilizado las siguientes librerías que permiten la integración de la interfaz gráfica, la comunicación por red y el procesamiento de datos:

- **Tkinter:** Librería estándar de Python encargada de la construcción de la interfaz gráfica de usuario (GUI) y la gestión del bucle de eventos principal.
- **Socket:** Módulo fundamental para la comunicación de red, permitiendo la creación de sockets TCP/IP para el intercambio de datos con el servidor Java.
- **Threading:** Utilizada para la implementación de la concurrencia, permitiendo que la recepción de datos ocurra en un hilo secundario sin bloquear la interfaz.
- **Pillow (PIL):** Librería externa empleada para el procesamiento de imágenes en el módulo MNIST, encargada del redimensionado y la normalización de píxeles.
- **OS:** Librería del sistema utilizada para la gestión de rutas y archivos locales.

Módulo	Responsabilidad
cliente_main.py	Punto de entrada, inicialización y configuración inicial
tcp_client_50.py	Gestión de conexión TCP, envío/recepción de mensajes
main_menu_cliente.py	Interfaz de menú principal, selector de modelos
form_mnist.py	Formulario para predicción de dígitos MNIST
form_w2v.py	Formulario para analogías y búsqueda Word2Vec

Table 6: Módulos principales del cliente en Python

4.2.1 Capa de Transporte: tcp_client_50.py

Este módulo encapsula la lógica de *sockets* de bajo nivel. Utiliza un hilo secundario (**Daemon Thread**) para escuchar las respuestas del servidor sin bloquear el *Main Loop* de la interfaz gráfica. Implementa

un protocolo basado en líneas de texto (UTF-8) para asegurar la compatibilidad multiplataforma. Este módulo es el núcleo de la comunicación TCP. Gestiona la conexión y la transmisión de mensajes.

Inicialización

```
1 class TCPClient50:
2     DEFAULT_SERVER_PORT = 4444
3
4     def __init__(self, ip, port=DEFAULT_SERVER_PORT,
5                 callback=None):
6         self.server_ip = ip
7         self.server_port = port
8         self.listeners = []
9         self.callback = callback
10        self.running = False
11        self.connected = False
12        self.socket = None
13        self.lock = threading.Lock()
```

Conexión: run()

```
1 def run(self):
2     self.running = True
3     try:
4         # Crea socket TCP/IP
5         self.socket = socket.socket(socket.AF_INET,
6                                     socket.SOCK_STREAM)
7
8         # Timeout de 5 segundos para conexión
9         self.socket.settimeout(5.0)
10        self.socket.connect((self.server_ip,
11                             self.server_port))
12
13        # Sin timeout después de conectarse
14        self.socket.settimeout(None)
15        self.connected = True
16        self.notify_connected()
17
18        # Lee líneas del servidor indefinidamente
19        with self.socket.makefile('r',
20                                  encoding='utf-8') as reader:
21            while self.running:
22                line = reader.readline()
23                if not line:
24                    break # Servidor cerró conexión
25                clean_line = line.strip()
26                if clean_line:
27                    self.notify_message(clean_line)
28
29    except Exception as e:
30        if self.running:
31            self.notify_error(f"Fallo de red: {e}")
32    finally:
33        self.stop_client()
34        self.notify_disconnected()
```

Envío de Mensajes: send_message()

```
1 def send_message(self, message):
2     if self.socket and self.connected:
3         try:
4             # Agrega salto de línea y codifica UTF-8
5             self.socket.sendall((message + "\n")
6                                 .encode('utf-8'))
7             return True
8         except:
9             self.connected = False
10            self.notify_error("Conexión perdida al enviar.")
11            return False
12     else:
13         self.notify_error("No hay conexión activa.")
14         return False
```

Patrón de Listeners

El cliente usa un patrón de observadores (listeners) para notificar eventos:

```
1 def add_listener(self, listener):
2     if listener not in self.listeners:
3         self.listeners.append(listener)
4
5 def notify_message(self, msg):
6     # Notifica a todos los listeners
7     for l in list(self.listeners):
8         if hasattr(l, 'message_received'):
9             l.message_received(msg)
10
11 def notify_connected(self):
12     for l in list(self.listeners):
13         if hasattr(l, 'on_connected'):
14             l.on_connected()
15
16 def notify_error(self, msg):
17     for l in list(self.listeners):
18         if hasattr(l, 'on_error'):
19             l.on_error(msg)
```

4.2.2 Procesamiento de Imágenes: form_mnist.py

A diferencia de implementaciones en otros lenguajes, la versión en Python aprovecha la librería **Pillow** para el tratamiento de datos. El cliente realiza el pre-procesamiento:

- Redimensionamiento a 28x28 píxeles.
- Conversión a escala de grises.
- Normalización de valores al rango [0, 1].
- Serialización a formato CSV para su transmisión.

Implementa el formulario para clasificación MNIST.

Procesamiento de Imagen

```
1 def enviar_al_servidor(self):
2     if not self.current_image:
3         messagebox.showwarning("Aviso",
4                                 "Carga una imagen primero.")
5         return
6
7     # Conversión MNIST: 28x28 píxeles, escala gris
8     img_mnist = self.current_image.convert("L")\
```

```

9         .resize((28, 28))
10
11     # Obtiene p xeles y normaliza a [0, 1]
12     pixels = list(img_mnist.getdata())
13     normalized = [p / 255.0 for p in pixels]
14
15     # Formato: 784 valores separados por comas
16     csv_data = ",".join([f"{p:.8f}"
17                          for p in normalized])
18
19     # Env a al servidor
20     self.tcp_client.send_message(
21         f"MNIST_PREDICT|{csv_data}")

```

Procesamiento de Respuesta

```

1 def _procesar_respuesta_seguro(self, message):
2     try:
3         if message.startswith("MNIST_RESULT|"):
4             parts = message.split("|")
5             if len(parts) >= 4:
6                 prediccion = parts[1]      # 0-9
7                 probabilidades = parts[2]  # prob por clase
8                 tiempo = parts[3]         # ms
9
10            # Actualiza interfaz
11            self.resultado_label.config(
12                text=f"Prediccion: {prediccion}",
13                fg="green")
14
15            self.prob_text.config(state="normal")
16            self.prob_text.delete("1.0", tk.END)
17            self.prob_text.insert(tk.END,
18                f"Probabilidades:\n{probabilidades}")
19            self.prob_text.config(state="disabled")

```

4.2.3 Procesamiento de Lenguaje Natural (form_w2v.py)

Este módulo implementa la interfaz para el modelo Word2Vec. Se encarga de capturar las analogías (A es a B como C es a X) y enviarlas mediante el comando `W2V_ANALOGY`. Maneja la recepción de listas de resultados y su visualización jerárquica en un área de texto con *scrolling*.

Implementa el formulario para Word2Vec.

Analogías

```

1 def resolver_analogia(self):
2     a = self.palabra_a.get().strip()
3     b = self.palabra_b.get().strip()
4     c = self.palabra_c.get().strip()
5
6     if not (a and b and c):
7         messagebox.showwarning("Aviso",
8             "Ingresa las tres palabras.")
9         return
10
11     # Env a: A - B + C = ?
12     self.tcp_client.send_message(
13         f"W2V_ANALOGY|{a}|{b}|{c}|10")

```

Búsqueda de Palabras Cercanas

```
1 def buscar_cercanas(self):
2     palabra = self.palabra_nearest.get().strip()
3
4     if not palabra:
5         messagebox.showwarning("Aviso",
6                                 "Ingresa una palabra.")
7         return
8
9     # Busca las 10 palabras m s similares
10    self.tcp_client.send_message(
11        f"W2V_NEAREST|{palabra}|10")
```

4.2.4 Orquestación y Lanzamiento (cliente_main.py y __main__.py)

El archivo `__main__.py` actúa como el punto de entrada estándar del sistema. `cliente_main.py` es el encargado de capturar los parámetros de red (IP y Puerto) y levantar el hilo demonio de red, asegurando que el cliente de red esté activo antes de mostrar el menú. Este es el punto de entrada de la aplicación. Realiza la inicialización y solicita los parámetros de conexión.

```
1 class Cliente50:
2     def __init__(self):
3         self.tcp_client = None
4
5     def iniciar(self):
6         root = tk.Tk()
7         root.withdraw()
8
9         # Solicita par metros
10        ip = simpledialog.askstring("IP",
11                                    "IP del servidor:", initialvalue="127.0.0.1")
12        port = simpledialog.askinteger("Puerto",
13                                       "Puerto del servidor:", initialvalue=4444)
14        name = simpledialog.askstring("Nombre",
15                                      "Nombre del cliente:", initialvalue="Cliente")
16
17        root.destroy()
18
19        # Crea cliente TCP y lo ejecuta en thread
20        self.tcp_client = TCPClient50(ip, port,
21                                       self.on_tcp_event)
22        threading.Thread(target=self.tcp_client.run,
23                          daemon=True).start()
24
25        # Crea men principal
26        menu = MainMenuCliente(self.tcp_client, name,
27                                ip, port)
28        menu.mainloop()
```

4.2.4.1 Función: iniciar()

- Crea ventana oculta para diálogos
- Solicita IP del servidor (default: 127.0.0.1)
- Solicita puerto (default: 4444)
- Solicita nombre del cliente
- Inicia cliente TCP en thread daemon
- Muestra menú principal

4.2.5 Navegación Central (main_menu_cliente.py)

Actúa como el despachador central de la aplicación. Utiliza el patrón de diseño *Observer* para recibir notificaciones desde el hilo de red y permitir la navegación segura entre los diferentes formularios sin perder la conexión con el servidor.

4.3 Implementación de Cliente - C++

Considerando de base las clases creadas con UI SWING y TcpClient50.java, se implementa la versión en C++ usando Qt como herramienta base de creación de interfaces moderno, esto con la finalidad de establecer comunicación con la ejecución de la clase TcpSeverThread50.java, dejando en claro la compatibilidad entre diferentes lenguajes de programación usando Sockets.

Existen dos formas de implementar interfaces en Qt en C++ al igual que Netbeans en Java, una de ellas es el uso de formularios <https://doc.qt.io/qt-6/designer-customizing-forms.html> en la opción designer donde se puede configurar directamente un archivo con extensión .ui que contiene tanto en estructura (XML) como en estilo (QSS) inline, siendo esta última semejante a CSS, mientras que la otra opción es utilizar directamente el framework que ofrece Qt para C++, algunas de estas librerías son:

```
#include <QApplication>
#include <QPushButton>
#include <QMessageBox>
#include <QCloseEvent>
#include <QScrollBar>
#include <QDateTime>
#include <QFileDialog>
#include <QFileInfo>
#include <QBuffer>
```

En la implementación de la interfaz se usó estas herramientas para poder agregar la funcionalidad de los formularios, es decir, para agregar el Backend.

En esta implementación en C++ se desarrolló dos ventanas de interfaces: mainformulariochatcliente.ui y mainmenuclient.ui como se ve en la imagen:

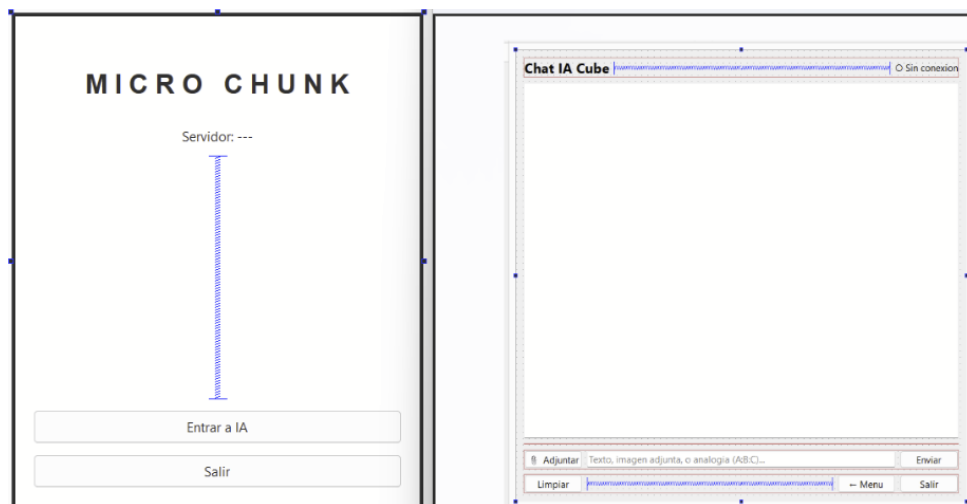


Figure 7: Interfaces de mainmenuclient.ui y mainformulariochatcliente.ui

Se debe de tomar en cuenta que los archivos 'headers'(archivos en extensión .h) no hacen más que definir funciones al igual que las interfaces en Java, y para la construcción (build) del proyecto se utiliza CMakeList.txt para poder definir recursos y versionamiento.

Para empezar con el flujo de comunicación entre el servidor (en java) y cliente (en C++) se toma de referencia el cliente construido en java, así que se crean métodos usando el framework de Qt en C++ y "QTcpSocket" para poder crear los siguientes métodos:

```

1 class TCPCClient50 : public QObject
2 {
3 public:
4     // Constructor y Destructor
5     explicit TCPCClient50(const QString &ip, int port, QObject *parent = nullptr);
6     ~TCPCClient50() override;
7
8     // Metodos principales de control y estado (El "Backend" de red)
9     void connectToServer();
10    void stopClient();
11    void sendMessage(const QString &message);
12    bool isConnected() const;
13
14    // Metodos de acceso (Getters)
15    QString serverIp() const;
16    int serverPort() const;
17
18 signals:
19     // Signals: Eventos que este cliente emite hacia la interfaz grafica
20     void messageReceived(const QString &message);
21     void connectedToServer();
22     void disconnectedFromServer();
23     void errorOccurred(const QString &errorMessage);
24
25 private slots:
26     // Slots: Funciones internas que reaccionan a los eventos del QTcpSocket
27     void onConnected();
28     void onDisconnected();
29     void onReadyRead();
30     void onSocketError(QAbstractSocket::SocketError socketError);
31 };

```

Listing 1: Métodos principales de la clase TCPCClient50

El formulario del chat interactúa con ambos de ambos modelos, de texto y de imagen, cuando escribes un mensaje interactuando con ellos o adjuntas una imagen y presionas 'enter', el sistema reconoce el patrón e interactúa con un modelo en base a ello, por tanto, si adjuntaste una foto (usando imágenes de MNIST) reconocerá el número en la imagen con una precisión calculada de probabilidad y si se escribe texto, el programa reconocerá dos tipos de estos, una con forma analógica ([Palabra1]:[Palabra2]:[Palabra3]) donde la palabra resultante es la palabra que debe conseguir darle sentido a la analogía, o cuando se envía solamente una palabra ([Palabra]) donde el modelo de texto mostrará palabras similares a esta para formar analogías. Ambos se envían al servidor donde recordemos que llevan etiquetas "MNIST-RESULT" o "W2V-NEAREST-RESULT" para que el servidor reconozca que tipo de modelo debe usar.

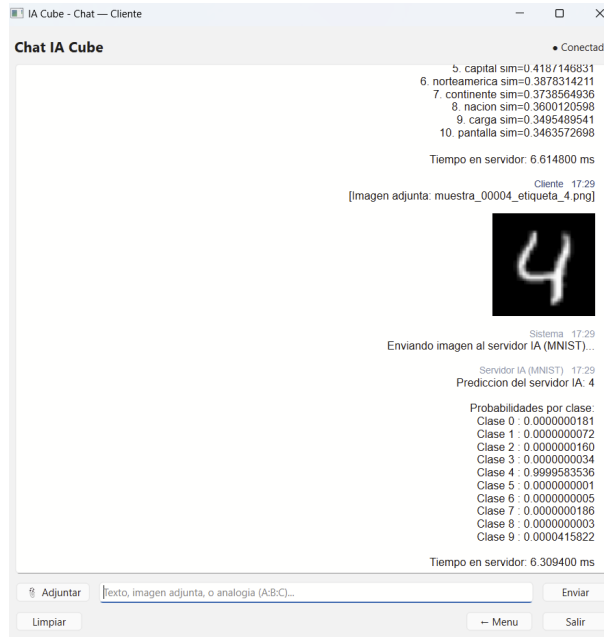


Figure 8: Interacción con el servidor con el formulario de mensajería

Además se muestra los métodos necesarios para que la comunicación sea posible a continuación:

```

1 class MainFormularioChatCliente : public QWidget
2 {
3 public:
4     // Constructor y Destructor
5     explicit MainFormularioChatCliente(TCPClient50 *tcpClient,
6                                         MainMenuClient *menu,
7                                         const QString &clientName,
8                                         QWidget *parent = nullptr);
9     ~MainFormularioChatCliente() override;
10
11 protected:
12     // Eventos del sistema operativo (Cierre de ventana)
13     void closeEvent(QCloseEvent *event) override;
14
15 private slots:
16     // Slots: Reacciones a los botones de la UI y a las seniales del TCPClient
17     void enviarMensaje();
18     void adjuntarImagen();
19     void limpiarChat();
20     void onMessageReceived(const QString &message);
21     void onConectado();
22     void onDesconectado();
23     void salirDelSistema();
24
25 private:
26     // Metodos internos de logica y renderizado visual
27     void appendMensaje(const QString &id, const QString &texto, bool esPropio);
28     void appendImagenPreview(const QString &id, const QPixmap &px, bool esPropio);
29
30     // Metodos de preprocesamiento para la Inteligencia Artificial
31     void enviarComoMNIST(const QImage &img);
32     void enviarComoW2V(const QString &texto);
33 };

```

Listing 2: Métodos principales de la clase MainFormularioChatCliente

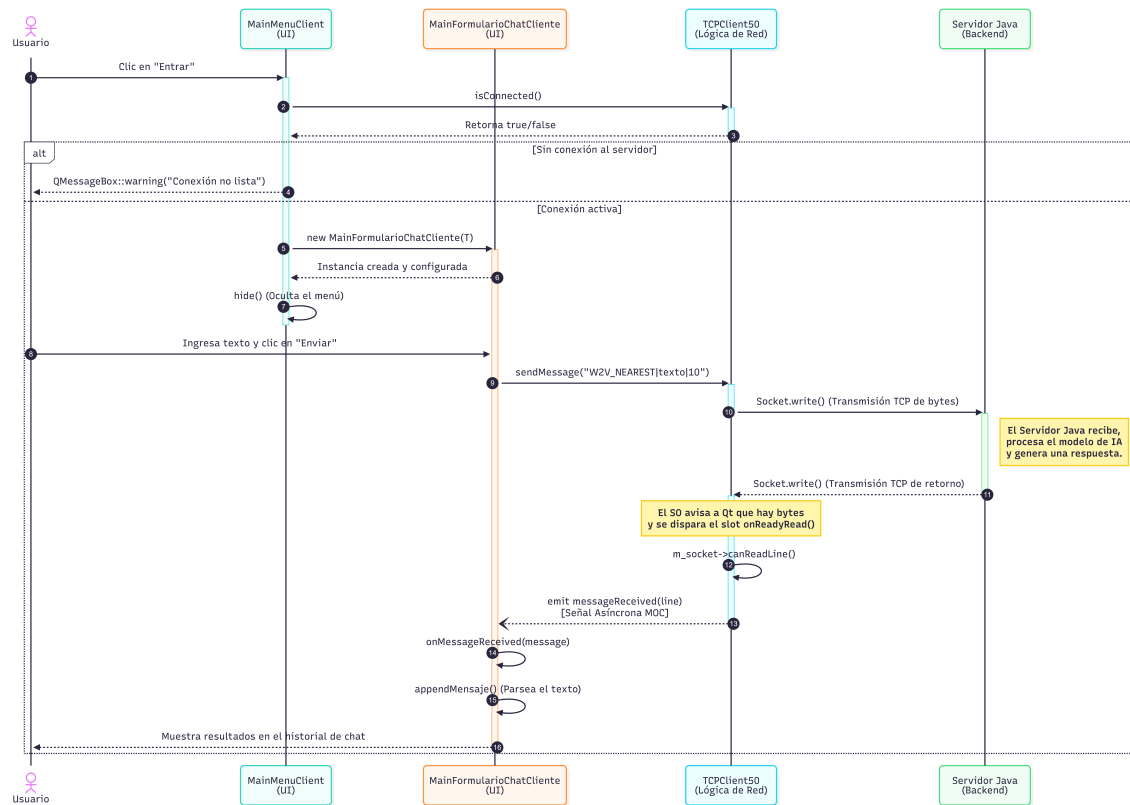


Figure 9: Diagrama de secuencia de la implementación de Cliente en C++

5 Pruebas y Resultados

En esta sección se presentan los resultados de las diversas pruebas a la aplicación. Las pruebas ejecutadas son:

- Paralelo, para ello se despliega en una PC el servidor y 3 clientes de diferentes códigos.
- Distribuido, en el cual se despliega 1 servidor en una Raspberry pi 4B y 3 clientes en Raspberry pi 4B.

5.1 Resultados de pruebas en máquina en Paralelo

En esta parte se trabajó con una PC, con sistema operativo usado será Windows 10, un procesador Intel Core i7-7700 @ 3.60Hz, con una RAM de 16GB, el equipo posee 4 de núcleos físicos y 8 hilos de ejecución. La IP que está trabajando en una red local para las pruebas es 192.168.0.164, y el puerto que se conectará a los clientes es el 4444.

Se presenta el despliegue del servidor en el lenguaje java (Fig. 10). Este espera que se conecten los clientes respectivos.

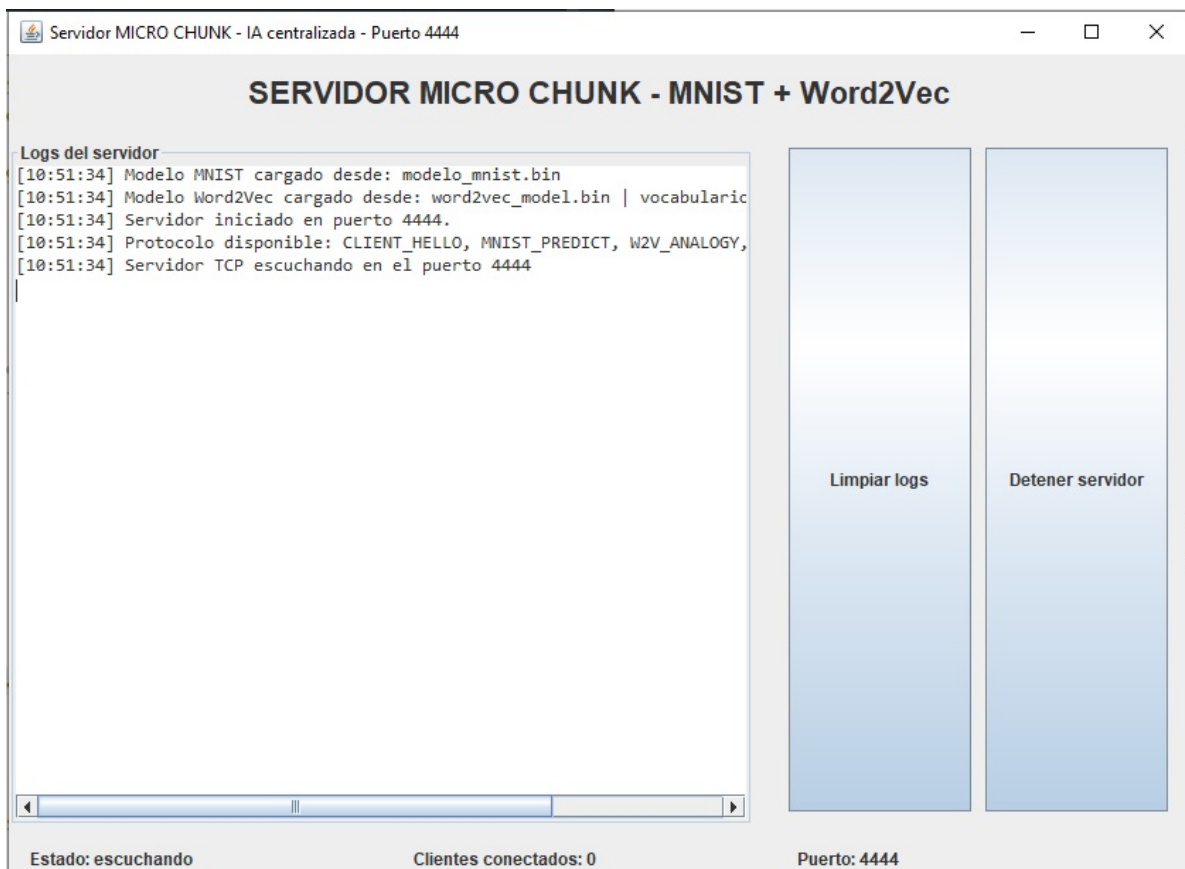


Figure 10: Vista del despliegue del servidor para las pruebas de paralelo

A continuación se hace el despliegue de tres clientes, el primero en java, el segundo en python y el tercero en C++.

El primer despliegue sera del cliente implementado en Java (Fig. 11).

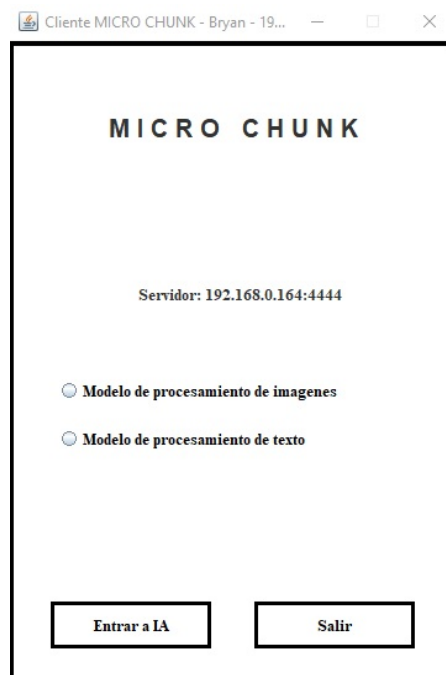


Figure 11: Vista del despliegue del Cliente en Java

El segundo despliegue sera del cliente implementado en Python (Fig. 12).



Figure 12: Vista del despliegue del Cliente en Python

El tercer despliegue sera del cliente implementado en C++(Fig. 13).

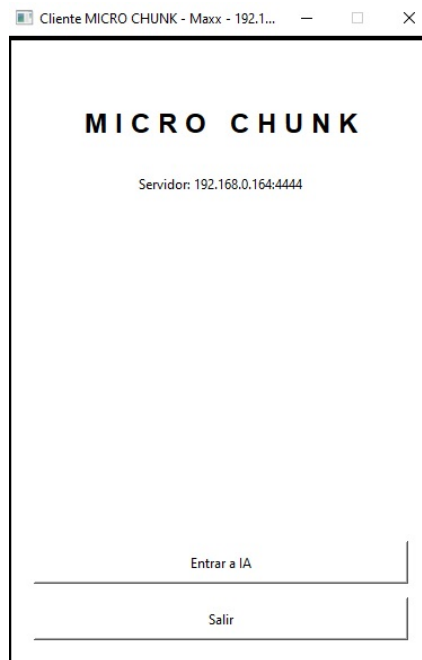


Figure 13: Vista del despliegue del Cliente en C++s

Finalmente se muestra el servidor conectados simultaneamente a los 3 clientes en una misma PC, cada cliente es un hilo(Fig. 14).

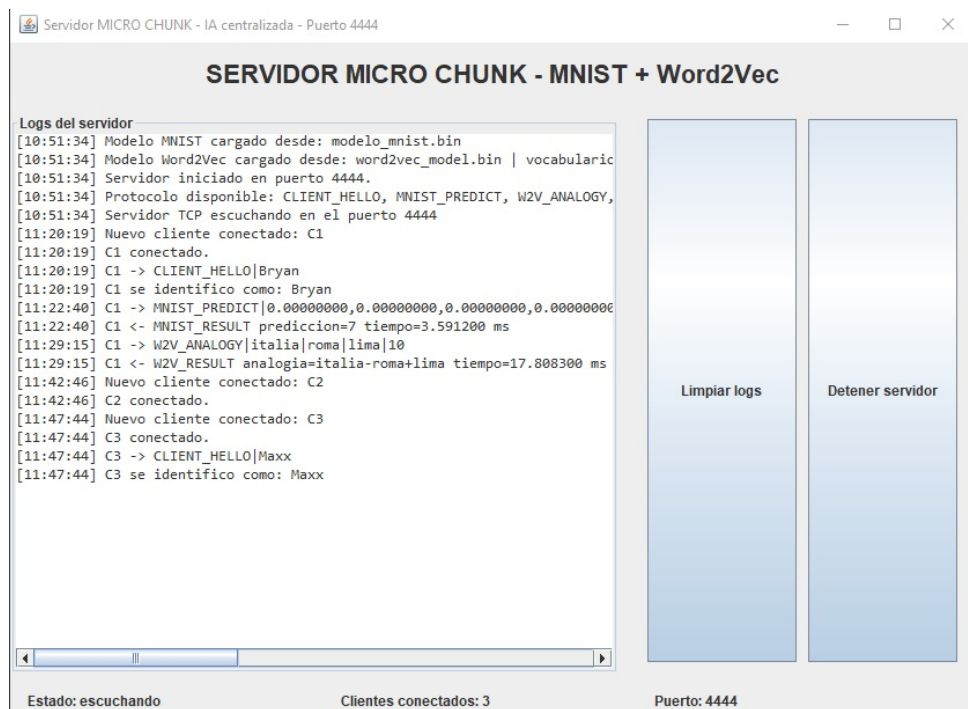


Figure 14: Vista del despliegue del servidor y los tres clientes conectados

5.2 Resultados de prueba en maquinas Distribuidos

En esta parte se tabajo se trabajo con 4 Raspberry Pi 4B, con sistema operativo Debian GNU/Linux 13 (trixie) aarch64, con una RAM de 8GB. Se usa 1 Raspberry Pi como servidor y los tres restantes como cliente.

Se presenta el despliegue en la primera Raspberry Pi 4B, el cual tiene la IP 192.168.0.133 y escuchara en el puerto 4444 (Fig. 15).

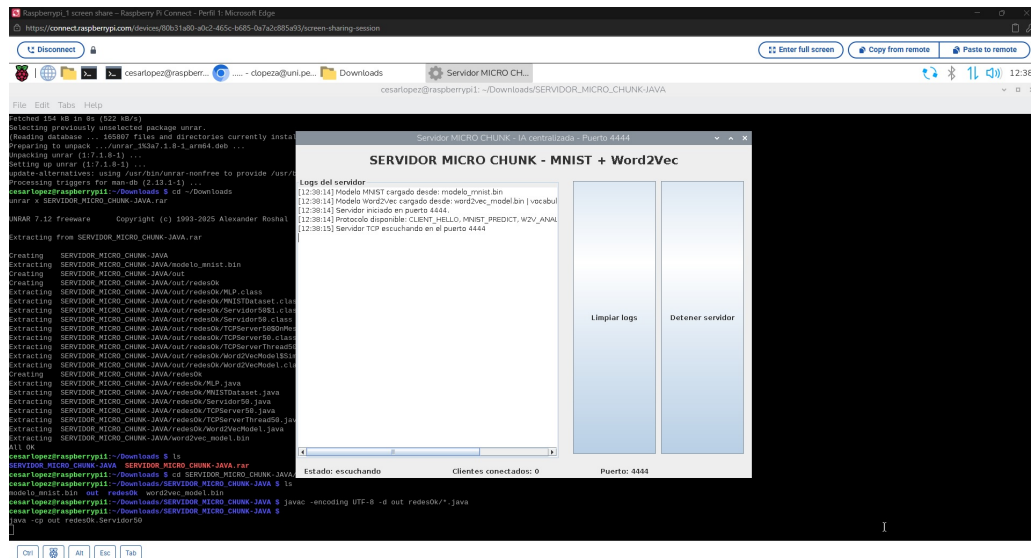


Figure 15: Vista del despliegue del servidor en una RaspBerry Pi4B

Se despliega el primer cliente en una Raspberry Pi 4B de IP 192.168.0.75, el cual esta implementado en Java (Fig. 16).

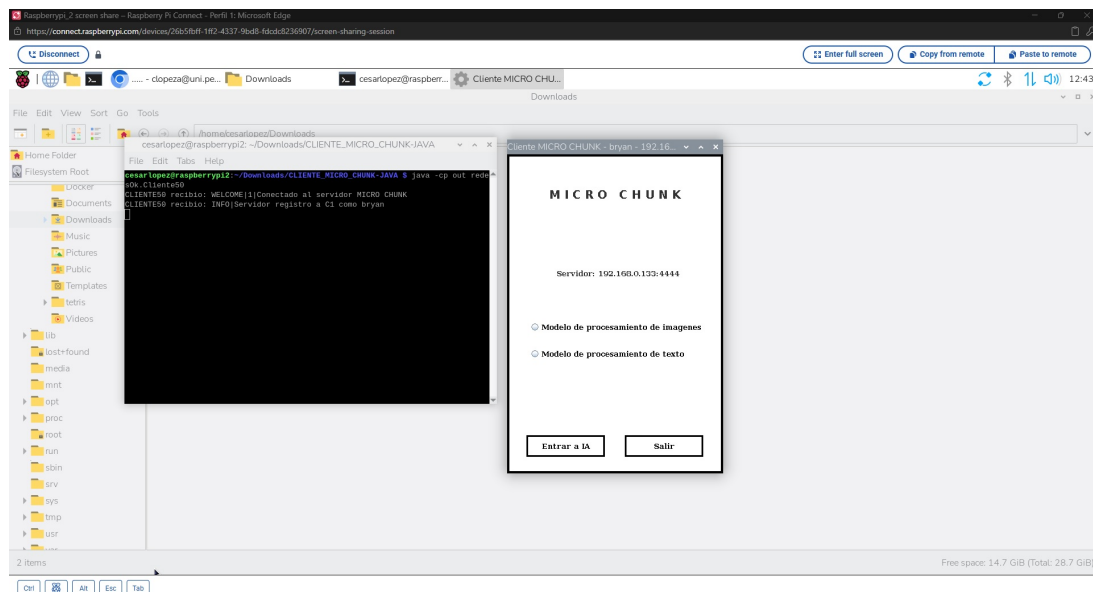


Figure 16: Vista del despliegue del cliente implementado en Java

Se despliega el segundo cliente en una Raspberry Pi 4B de IP 192.168.0.82, el cual esta implementado en Python (Fig. 17).

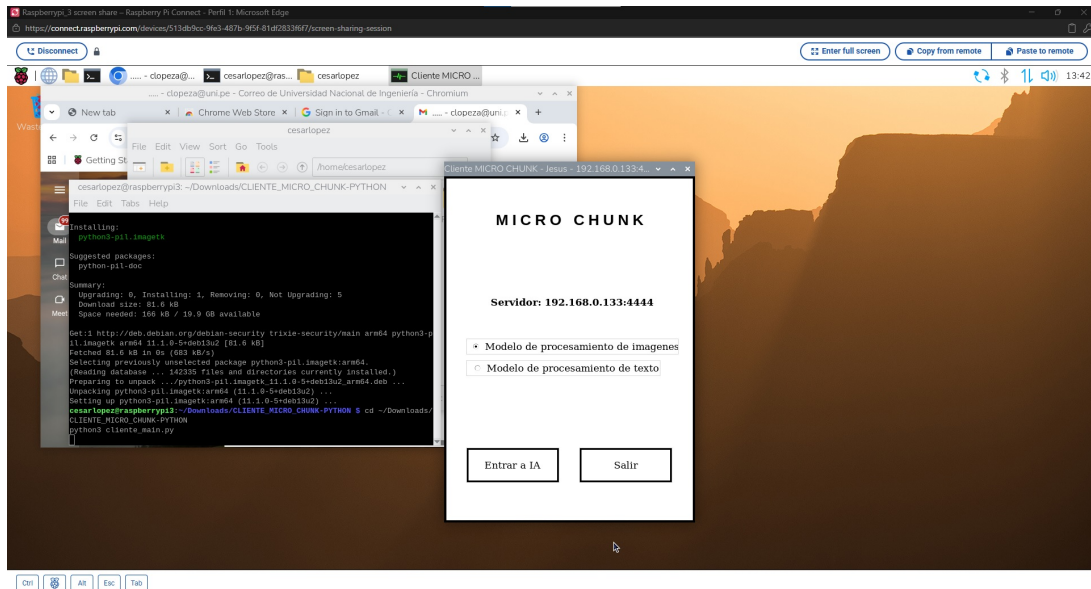


Figure 17: Vista del despliegue del cliente implementado en Python

Se despliega el segundo cliente en una Raspberry Pi 4B de IP 192.168.0.82, el cual esta implementado en C++ (Fig. 18).

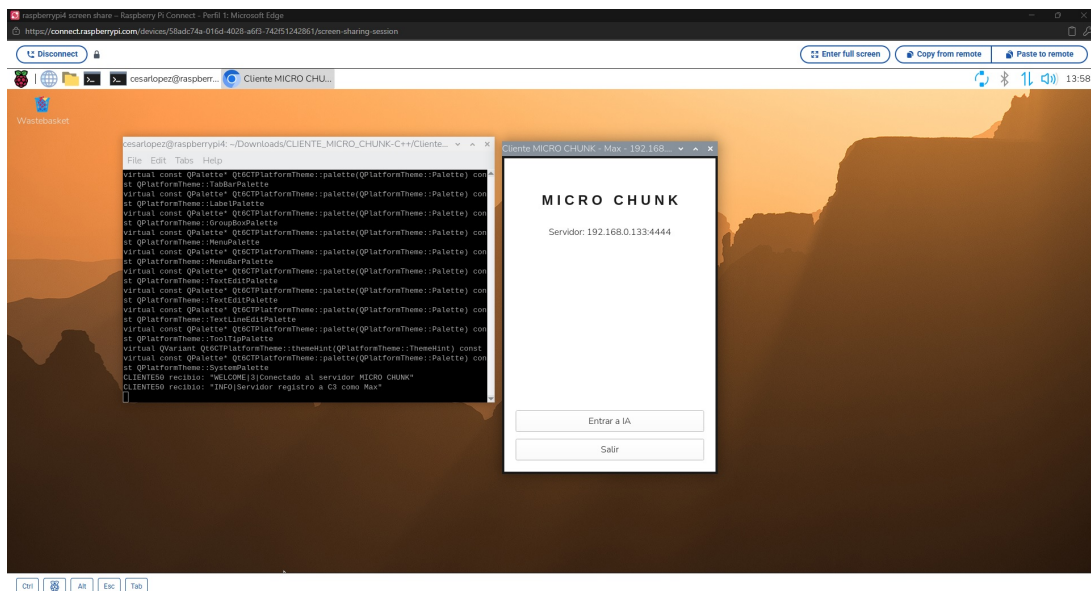


Figure 18: Vista del despliegue del cliente implementado en C++

Se muestra la conexión que del servidor con los tres cliente implementados en lenguajes diferentes y cada cliente en una computadora diferente (Fig. 19).

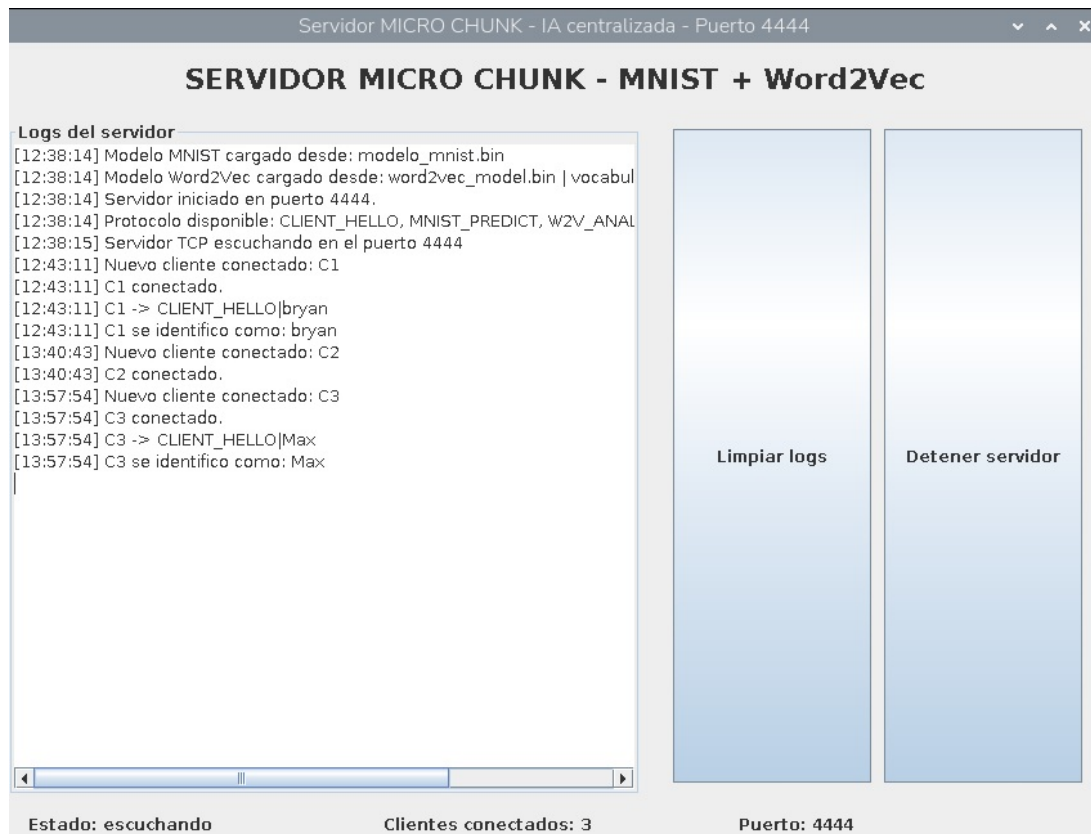


Figure 19: Vista del despliegue del servidor y los tres clientes conectados

6 Conclusiones

- Se implementó una versión funcional de Open AI Cubes con dos dimensiones: texto e imagen.
- El sistema usa una arquitectura cliente-servidor distribuida, donde el servidor centraliza los modelos y el cliente actúa como consumidor ligero.
- La concurrencia se logra mediante un hilo por cliente, permitiendo múltiples conexiones simultáneas.
- El módulo de imagen usa una red MLP 784–100–50–10 con resultados funcionales sobre MNIST y precisión de prueba de 97.06%.
- El módulo de texto usa embeddings Word2Vec Skip-gram para similitud semántica y analogías.
- La arquitectura es extensible: nuevos cubos pueden agregarse mediante nuevos modelos, comandos de protocolo y formularios de cliente.

References

1. E. W. Dijkstra, *A Note on Two Problems in Connexion with Graphs*, Numerische Mathematik, vol. 1, pp. 269–271, 1959.
2. T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.
3. B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes y D. Lea, *Java Concurrency in Practice*, Addison-Wesley, 2006.
4. M. Herlihy y N. Shavit, *The Art of Multiprocessor Programming*, Morgan Kaufmann, 2012.
5. D. E. Rumelhart, G. E. Hinton y R. J. Williams, *Learning Representations by Back-Propagating Errors*, Nature, vol. 323, pp. 533–536, 1986.
6. Y. LeCun, L. Bottou, Y. Bengio y P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.
7. I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*, MIT Press, 2016.
8. T. Mikolov, K. Chen, G. Corrado y J. Dean, *Efficient Estimation of Word Representations in Vector Space*, arXiv preprint arXiv:1301.3781, 2013.
9. T. Mikolov, I. Sutskever, K. Chen, G. Corrado y J. Dean, *Distributed Representations of Words and Phrases and their Compositionality*, Advances in Neural Information Processing Systems, vol. 26, 2013.
10. Y. Goldberg y O. Levy, *word2vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method*, arXiv preprint arXiv:1402.3722, 2014.
11. X. Rong, *word2vec Parameter Learning Explained*, arXiv preprint arXiv:1411.2738, 2014.
12. O. Levy y Y. Goldberg, *Neural Word Embedding as Implicit Matrix Factorization*, Advances in Neural Information Processing Systems, vol. 27, 2014.
13. Y. Goldberg, *Neural Network Methods for Natural Language Processing*, Morgan & Claypool Publishers, 2017.
14. S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Pearson, 2009.

15. Y. LeCun y C. Cortes, *MNIST handwritten digit database*, 2010.
16. J. Leskovec y A. Krevl, *SNAP Datasets: Stanford Large Network Dataset Collection*, 2014.
17. Jeff Calder, *Math 5467: Introduction to the Mathematics of Image and Data Analysis*, University of Minnesota, School of Mathematics, May 10, 2022.
18. Thomas Cormen y Charles Leiserson, *Introduction to Algorithms Third Edition*.
19. Jose Tomas Palma, *Programacion Concurrente*.